

THE PRACTICAL EDITION

GIT & GITHUB

COMMAND FIELD GUIDE



Build confidence from first commit to recovery.

Commands, mental models, safe undo paths, and step-by-step fixes for the errors developers meet most often.

THE MEMORY LOOP

1

EDIT

working tree

2

STAGE

git add

3

SAVE

git commit

4

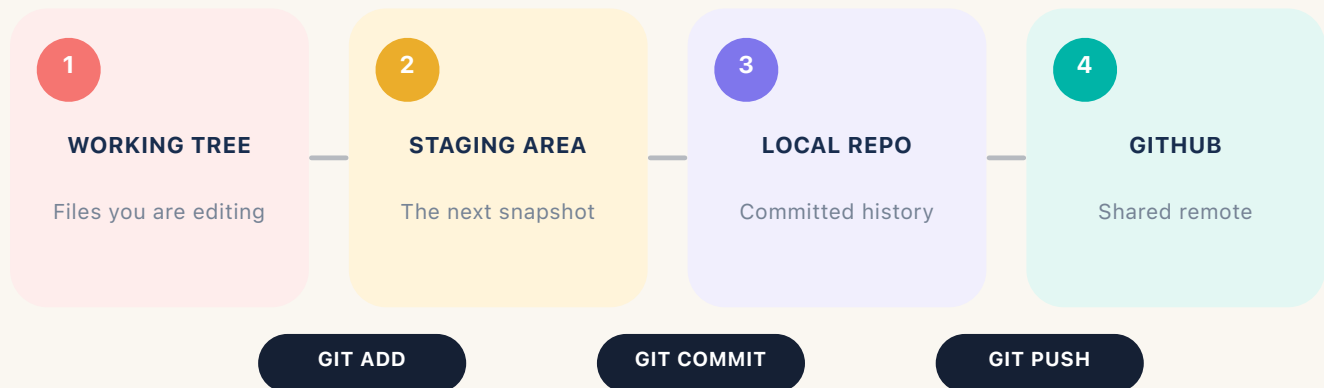
SHARE

git push

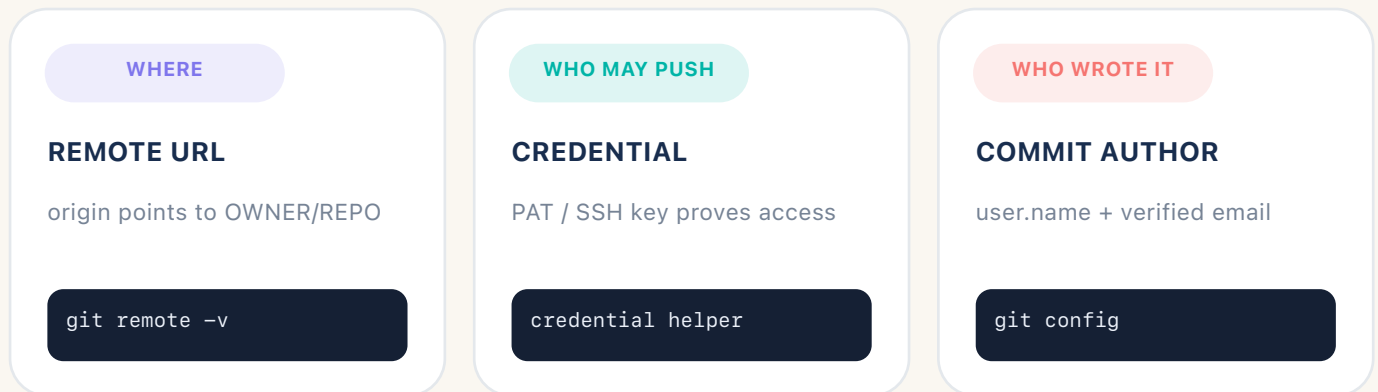
How Git thinks

Four places, three identities, one repeatable mental model.

THE FOUR ZONES



THREE IDENTITIES PEOPLE CONFUSE



SAFETY LANGUAGE



GREEN • inspect or add

status, diff, log, fetch, add, commit. These are routine and recoverable.



AMBER • rewrites locally

amend, rebase, reset. Pause and confirm whether commits were already pushed.



RED • can discard or expose

reset --hard, clean -fd, force push, committing secrets. Make a backup branch and read twice.

MEMORY RULE

Status tells you where you are. Diff tells you what changed. Log tells you how you got there.

Setup once, verify often

Use global identity for your default account; use local identity for exceptions.

CONFIRM THE INSTALLATION

Version & help

SAFE

```
git --version
git help <command>
```

Check Git and open the manual for any command, for example: `git help switch`.

Inspect configuration

SAFE

```
git config --list --show-origin
```

See each setting and the file it came from. Essential when global and local settings disagree.

DEFAULT IDENTITY

Set your main account

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

The email should be verified on the GitHub account that should receive commit attribution.

ONE REPOSITORY USES ANOTHER ACCOUNT

Override only inside the current repository

```
git config --local user.name "Other Name"
git config --local user.email "verified@example.com"
```

Local settings live in `.git/config` and override global settings only for this repository.

VERIFY BEFORE THE FIRST COMMIT

Read the effective identity

```
git config user.name
git config user.email
```

No `--global` means: show the value Git will actually use here. Fix it before committing.

Check the latest author

```
git log -1 --format='%an <%ae>'
```

A token controls authentication. This email controls commit attribution.

Start a repository

Choose one path: clone an existing project, or publish a local folder.

PATH A • CLONE SOMETHING ALREADY ON GITHUB

Clone, enter, inspect

```
git clone https://github.com/OWNER/REPO.git
cd REPO
git status
```

Clone creates the folder, downloads history, sets origin, and checks out the default branch.

PATH B • PUBLISH AN EXISTING LOCAL FOLDER

1

Initialize

git init

2

Inspect before staging

git status

3

Stage the intended files

git add .

4

Create the first snapshot

git commit -m "Initial commit"

5

Connect and publish

git branch -M main → remote add → push

Complete publish sequence

```
git remote add origin https://
github.com/OWNER/REPO.git
```

-u records the upstream, so later you can usually run only git push and git pull.

Verify the connection

```
git remote -v
git branch -vv
```

Confirm the destination URL and which remote branch the local branch tracks.



Before git add .

Create .gitignore first. Never stage .env, API tokens, private keys, dependency folders, build output, or OS/editor clutter.



Remote already exists?

Use git remote set-url origin <URL>. Do not add another origin with the same name.

The daily loop

Sync → branch → edit → inspect → stage → commit → push.

1

SYNC`pull --rebase`

2

BRANCH`switch -c`

3

EDIT`your files`

4

CHECK`status + diff`

5

SAVE`add + commit`

6

SHARE`push`

A CLEAN FEATURE WORKFLOW

Run this rhythm

```
git switch main
git pull --rebase
git switch -c feature/short-name
# edit files
git status
git diff
git add -p
git commit -m "Describe the outcome"
git push -u origin feature/short-name
```

Small focused commits are easier to review, revert, merge, and understand six months later.

COMMIT MESSAGE PATTERN

<verb> <useful outcome>

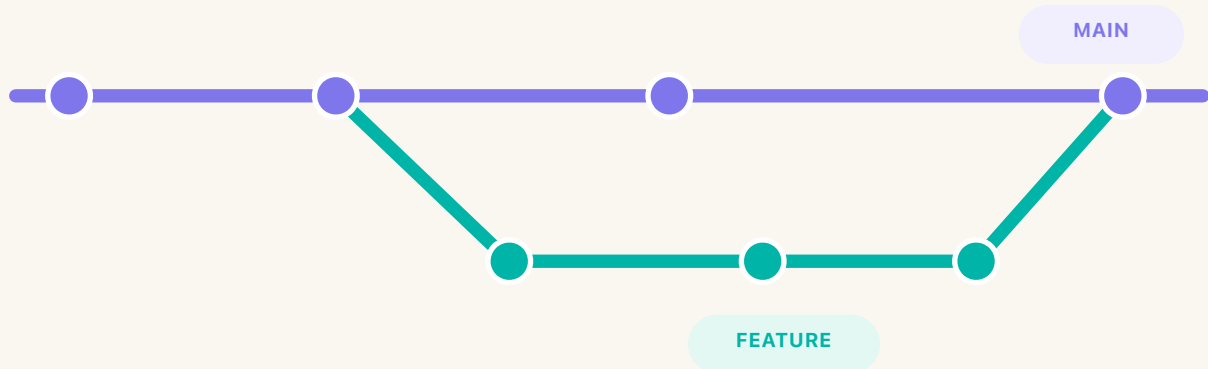
Add contact-form validation • Fix mobile navigation overlap

**Before every push**

Run tests, review git status, and check git diff --staged. Push only what you intended to publish.

Branches without confusion

A branch is a movable label pointing at a commit.



List branches

```
git branch
git branch -a
git branch -vv
```

Local, all, then verbose with upstream and last commit.

Create and switch

```
git switch -c feature/name
```

Creates the branch at your current commit and moves HEAD to it.

Switch branches

```
git switch <branch>
git switch -
```

The dash returns to the previously checked-out branch—the fastest context toggle.

Merge finished work

```
git switch main
git merge feature/name
```

Always switch to the branch that should receive the changes before merging.

Delete after merge

```
git branch -d feature/name
git push origin --delete feature/name
```

-d protects unmerged work. The second command deletes the remote branch.

Clean stale remote labels

```
git fetch --prune
```

Downloads remote updates and removes local references to deleted remote branches.

→ Branch naming

Use short intent-based names: feature/contact-form, fix/mobile-nav, docs/setup-guide.

Inspect before you act

The best Git habit is asking the repository what is true.

Compact status

START

```
git status -sb
```

Shows branch, upstream position, and changed files in a compact form.

Unstaged changes

```
git diff
```

What changed in the working tree but has not been staged.

Staged changes

```
git diff --staged
```

Exactly what the next commit will contain. Review before committing.

One file or commit

```
git diff -- <file>
git show <commit>
```

Narrow the view to a file or inspect a saved snapshot.

SEE THE STORY

Readable graph

```
git log --oneline --graph --decorate --all
```

A compact map of commits, branch labels, merges, and remote-tracking branches.

Search commits

```
git log --grep="login"
git log -- <file>
```

Search commit messages, or follow the history of a particular file.

Who changed this line?

```
git blame <file>
```

Shows the last commit touching each line. Use it to find context, not to assign blame.

THREE QUESTIONS

Where am I?

status

What changed?

diff

How did I get here?

log

Stage and commit precisely

The staging area is your editor for the next snapshot.

Stage selected files

```
git add <file>
git add <folder>/
```

Prefer explicit paths when a project contains unrelated changes.

Stage interactively

BEST

```
git add -p
```

Review each hunk and choose what belongs in this commit. A precision superpower.

Unstage safely

```
git restore --staged <file>
```

Moves a file out of staging without discarding your edits.

Commit the snapshot

```
git commit -m "Useful outcome"
```

A commit stores the staged snapshot locally; it does not upload anything.

CORRECT THE LATEST LOCAL COMMIT

Add a forgotten file

CAUTION

```
git add <file>
git commit --amend --no-edit
```

Replaces the latest commit. Use before pushing, or coordinate before rewriting shared history.

IGNORE INTENTIONALLY

Typical .gitignore entries

```
.env
node_modules/
dist/
.DS_Store
```

Ignore generated, local, secret, or machine-specific files—not source code needed to build the

Already tracked?

```
git rm --cached <file>
git commit -m "Stop tracking local"
```

.gitignore does not untrack an existing file. --cached keeps the local copy.



Commit ≠ backup until pushed

A local commit survives edits, but not disk loss. Push important work to a trusted remote.

Remotes and synchronization

Fetch observes, pull integrates, push publishes.

FETCH

Downloads remote facts without changing your files

PULL

Fetches, then integrates into the current branch

PUSH

Uploads local commits to a remote branch

Inspect remotes

```
git remote -v  
git remote get-url origin
```

Verify both the remote name and exact destination before a first push or account switch.

Change origin

```
git remote set-url origin <URL>
```

Use this when origin already exists or points at the wrong repository.

Download only

```
git fetch origin  
git fetch --all --prune
```

Safe way to update your view of remote branches before deciding how to integrate.

Update current branch

```
git pull --rebase
```

Replays your unpublished local commits after fetched commits, keeping a linear branch.

Publish a new branch

```
git push -u origin <branch>
```

Sets upstream once. Later, git push is enough on that branch.

Tags and releases

```
git tag v1.0.0  
git push origin v1.0.0
```

Tags mark important commits. Push the tag explicitly so GitHub receives it.



Force with a seat belt

If you intentionally rewrote your own pushed branch, prefer `git push --force-with-lease`. It refuses to overwrite remote work you have not seen.

Pause work with stash

Temporarily shelve unfinished edits without creating a commit.

THE CONTEXT-SWITCH RECIPE

1 Save

```
git stash push -m "WIP: contact form"
```

2 Switch

```
git switch main
```

3 Return

```
git switch -
```

4 Restore

```
git stash pop
```

KNOW WHAT IS STORED

List stashes

```
git stash list
```

Newest is `stash@{0}`. Messages make the list understandable.

Inspect one

```
git stash show -p stash@{0}
```

Review the patch before restoring it elsewhere.

Apply but keep

```
git stash apply stash@{0}
```

Restores changes without deleting the stash—safer when experimenting.

Delete intentionally

CHECK

```
git stash drop stash@{0}
```

Removes one stash. Avoid `git stash clear` unless you truly want to delete all.



Untracked files

`git stash push -u -m "WIP"` includes untracked files. Ignored files remain excluded unless you use `-a`.

Undo without panic

Choose the command based on where the change currently lives.

UNSTAGED EDIT

`git restore <file>`

Discards working-tree edits

STAGED FILE

`git restore --staged <file>`

Keeps edits; removes from staging

LOCAL COMMIT

`git reset --soft HEAD~1`

Removes commit; keeps changes staged

SHARED COMMIT

`git revert <commit>`

Adds a new inverse commit

LOST POINTER

`git reflog`

Finds recent HEAD positions

RECOVERY LADDER

1 • Inspect

`git status → git diff → git log --online --graph --all`

2 • Create a rescue label

`git branch rescue-before-fix` — a cheap pointer before rewriting anything.

3 • Recover from reflog

`git reflog → identify the good hash → git switch -c rescue <hash>`

DANGER

`git reset --hard` discards tracked working changes. `git clean -fd` deletes untracked files.

Resolve merge conflicts

A conflict means Git needs a human decision—not that anything is broken.

```
<<<<<<< HEAD
your current branch
=====
incoming branch
>>>>>>> feature/name
```

Choose one version or combine both. Then remove every marker line.

STEP BY STEP

1

Find conflicted files

`git status`

2

Open each file

Search for `<<<<<<<`, `=====`, and `>>>>>>>`

3

Edit the final version

Keep the correct lines; delete all markers

4

Mark resolved

`git add <resolved-file>`

5

Continue

`git commit` OR `git rebase --continue`

6

Verify

Run tests, then `git status` and `git log`

BEST PRACTICE

Commit or stash unrelated work before merging so the conflict stays understandable.

NEED TO STOP?

Abort a merge

```
git merge --abort
```

Return to the state before the merge began.

Abort a rebase

```
git rebase --abort
```

Return to the state before the rebase began.

403: permission denied

The destination URL and the authenticating account are different settings.



What the error means

GitHub received valid credentials, but that account cannot write to OWNER/REPO. A remote URL selects where; a PAT or SSH key proves who.

DIAGNOSE

Confirm destination

```
git remote -v
```

Check spelling of OWNER and REPO for both fetch and push.

Confirm commit identity

```
git config user.email
```

Useful for attribution, but changing this alone will not fix authentication.

HTTPS FIX FOR ONE REPOSITORY/ACCOUNT

Scope credentials to the URL path

```
git config --local credential.https://github.com.useHttpPath true
git remote set-url origin https://ACCOUNT@github.com/OWNER/REPO.git
git push -u origin main
```

When prompted for Password, paste a fine-grained personal access token—not the GitHub password.

FINE-GRAINED TOKEN CHECKLIST

- ✓ Create it while signed into the intended account
- ✓ Resource owner: the repository owner
- ✓ Repository access: only the required repository
- ✓ Contents permission: Read and write
- ✓ Set an expiration; copy it once; never commit it



Token safety

If a token appears in chat, a screenshot, a remote URL, shell history, or committed code: revoke it immediately and

Three common errors

Read the exact message, preserve work, then choose the smallest fix.

1 • NON-FAST-FORWARD

Remote has commits you do not have

```
git fetch origin
git log --oneline --graph --all
git pull --rebase origin main
git push
```

Fetch and inspect first. Resolve any rebase conflict, stage the fix, then run `git rebase --continue`.

2 • DETACHED HEAD

Save work on a real branch

```
git switch -c rescue/my-work
```

Detached HEAD means you checked out a commit rather than a branch. Creating a branch keeps new commits reachable.

3 • ORIGIN ALREADY EXISTS / WRONG DESTINATION

Inspect, then replace the URL

```
git remote -v
git remote set-url origin https://github.com/OWNER/REPO.git
git remote -v
```

Use `set-url` instead of trying to add a second remote named `origin`.

QUICK ORIENTATION

Not a Git repository

```
pwd
ls -la
git status
```

Move into the project folder containing `.git`. Use `git`

Branch name mismatch

```
git branch
git branch -M main
```

Check whether your branch is `main`, `master`, or something else before pushing.

Mistakes that need special care

Secrets, identity, and tracked files require different remedies.

A SECRET WAS COMMITTED

1 • Revoke or rotate first

Assume the secret is compromised. Revoke the PAT/API key/password at its provider before editing Git history.

2 • Remove from the current tree

Add the file to .gitignore, then use `git rm --cached <file>`, commit, and push.

3 • Historical cleanup is separate

Deleting the latest file does not erase old commits. Coordinate history rewriting with collaborators; then rotate again if needed.

WRONG NAME OR EMAIL

Fix future commits here

```
git config --local user.name "Your Name"
git config --local user.email "verified@example.com"
```

Use an email verified on the intended GitHub account. Local scope keeps other projects unchanged.

Fix latest unpublished commit

CAUTION

```
git commit --amend --reset-author --no-edit
```

This replaces the latest commit. If it was pushed, coordinate before using force-with-lease.

FILE IGNORED BUT STILL APPEARS

Stop tracking; keep local copy

```
git rm --cached <file>
git commit -m "Stop tracking local file"
```

Then verify with `git status` and push the commit.

Pocket index

The smallest set of commands that handles most everyday work.

ORIENT

```
git status -sb
```

NEW BRANCH

```
git switch -c <name>
```

SEE EDITS

```
git diff
```

PREVIOUS

```
git switch -
```

SEE STAGED

```
git diff --staged
```

SYNC

```
git pull --rebase
```

STAGE

```
git add -p
```

PUBLISH

```
git push -u origin <branch>
```

SAVE

```
git commit -m "..."
```

SHELVE

```
git stash push -m "WIP"
```

HISTORY

```
git log --oneline --graph --all
```

RECOVER

```
git reflog
```

TEN RULES WORTH REMEMBERING

1. Start with status.
2. Review diff before add.
3. Review --staged before commit.
4. Commit one outcome at a time.
5. Pull before starting shared work.
6. Branch before experimenting.
7. **Never commit secrets.**
8. Revert shared history; reset local history.
9. **Prefer --force-with-lease over --force.**
10. Reflog is the emergency map.

OFFICIAL REFERENCES

git-scm.com/docs • git-scm.com/book • docs.github.com/get-started/using-git